Computers & Security 124 (2023) 102996

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Towards a fair comparison and realistic evaluation framework of android malware detectors based on static analysis and machine learning

Borja Molina-Coronado[a,*], Usue Mori[b], Alexander Mendiburu[a], Jose Miguel-Alonso[a]

[a] Department of Computer Architecture and Technology, University of the Basque Country UPV/EHU, Ps. Manuel Lardizabal 1, Donostia-San Sebastián, Gipuzkoa 20018, Spain

[b] Department of Computer Science and Artificial Intelligence, University of the Basque Country UPV/EHU, Ps. Manuel Lardizabal 1, Donostia-San Sebastián, Gipuzkoa 20018, Spain

ARTICLE INFO

Article history:
Received 24 May 2022
Revised 19 September 2022
Accepted 26 October 2022
Available online 28 October 2022

Keywords:
Android malware detection
Machine learning
Mobile security
Experimental analysis
Static analysis

ABSTRACT

As in other cybersecurity areas, machine learning (ML) techniques have emerged as a promising solution to detect Android malware. In this sense, many proposals employing a variety of algorithms and feature sets have been presented to date, often reporting impresive detection performances. However, the lack of reproducibility and the absence of a standard evaluation framework make these proposals difficult to compare. In this paper, we perform an analysis of 10 influential research works on Android malware detection using a common evaluation framework. We have identified five factors that, if not taken into account when creating datasets and designing detectors, significantly affect the trained ML models and their performances. In particular, we analyze the effect of (1) the presence of duplicated samples, (2) label (goodware/greyware/malware) attribution, (3) class imbalance, (4) the presence of apps that use evasion techniques and, (5) the evolution of apps. Based on this extensive experimentation, we conclude that the studied ML-based detectors have been evaluated optimistically, which justifies the good published results. Our findings also highlight that it is imperative to generate realistic experimental scenarios, taking into account the aforementioned factors, to foster the rise of better ML-based Android malware detection solutions.

© 2022 Elsevier Ltd. All rights reserved.

1. Introduction

Over the past decade we have witnessed impressive advances in mobile devices. Along with the hardware, operating systems (OS) designed for the mobile market have experienced pairwise functionality improvements. With a market share near 72% as of the last quarter of 2021 (Statista, 2022), the Android platform is the leading mobile OS. It has an open source nature and is available for multiple processor architectures. These facts, along with the availability of a well documented development framework that enables a rich set of services (voice and image recognition, contactless payments, etc.), have contributed to the adoption of Android beyond smartphones (Alaa et al., 2017; Do et al., 2017; Macario et al., 2009).

Coupled with this growing popularity, the increasing attention paid by malware writers to this OS has highlighted the risks to which users are exposed (Xu et al., 2016b). Data stored on mobile devices is of vital importance, sensitive for users, and has become a valuable target for attackers. According to Kaspersky, adware and banking malware targeting these devices were two of the main security threats in 2020, even being detected in trusted app markets such as Google Play (Labs, 2021).

Aware of these problems, researchers have seen machine learning (ML) techniques as a promising solution for the implementation of Android malware detectors (Tan et al., 2015). ML methods leverage on app data to identify signals that are useful for detecting malware. To this end, malware detection can follow one of these strategies (or both in combination): (1) *anomaly-based* detection focuses on building profiles from goodware so that deviations from those profiles are flagged as dangerous; (2) *misuse-based* detection, instead, focuses on learning the characteristics of both malware and goodware in order to identify their presence in new apps (Molina-Coronado et al., 2020). Most of the research works on Android malware detection belong to the second group. More specifically, relying on supervised ML algorithms for their detection mechanisms (Faruki et al., 2014; Tam et al., 2017).

* Corresponding author.
E-mail address: borja.molina@ehu.eus (B. Molina-Coronado).

https://doi.org/10.1016/j.cose.2022.102996
0167-4048/© 2022 Elsevier Ltd. All rights reserved.

To perform the detection of malware using ML algorithms (either for misuse or anomaly-based systems), apps need to be preprocessed in order to extract the set of features that best describe their behavior. This task is performed using dynamic or static software analysis techniques. In *dynamic analysis*, the behavior of an app is monitored in a controlled environment (sandbox), where user and system interactions are simulated. *Static analysis* is based on the inspection of the files contained in the application package (APK) without needing to run the code. These techniques have their own advantages and drawbacks. On the one hand, through dynamic analysis, it is possible to access the code that is loaded and executed in runtime. However, the success of this analysis, in terms of coverage of code, greatly depends on the simulation mechanism and the absence of sandbox evasion instruments in the apps (Afianian et al., 2019). On the other hand, static analysis is able to evaluate all the information present in the APKs, but its success lies in the absence of evasion techniques such as obfuscation or dynamic loading of code (Moser et al., 2007). Both analysis approaches are complementary and can be combined for Android malware detection (Arshad et al., 2018; Spreitzenbarth et al., 2015).

One of the main difficulties faced by researchers when proposing, developing and testing Android malware detectors is the absence of a common and realistic evaluation framework. This framework should include appropriate and labeled datasets, essential for training and/or testing ML algorithms. In particular, and as we illustrate in this paper, most publicly available datasets are obsolete, only contain malware, repeated samples, or comprise an insufficient number of samples. In view of this, authors opt for building ad-hoc, custom datasets by downloading app samples from different sources and labeling them using tools such as VirusTotal[1]. This process is not only expensive, but also complicates the reproducibility and comparison of ML-based malware detection proposals.

The issue of reproducibility is aggravated by the unavailability of the code that implements the proposed methods, or by the omission in their respective publications of important details that allow their implementation. This is specially manifested for methods using ML algorithms that require tuning a large number of parameters in order to perform properly (Pendlebury et al., 2019), as this information is not often provided. The same is true for the evaluation procedures. In many cases, they are not clearly described or are designed assuming very optimistic scenarios (Allix et al., 2016a; Pendlebury et al., 2019).

The main objective of this study is to perform a fair comparison of Android malware detection proposals already published in the literature, shedding light about their actual effectiveness. Given the vast amount of proposals presented over the years, as well as the absence of common and realistic evaluation criteria, performing a fair comparison of methods is not a straightforward task. We have chosen 10 popular detectors based on static analysis that use different features and ML methods, and compared them under a common evaluation framework. In many cases, a re-implementation of the algorithms used in the detectors has been required due to the lack of the original authors' implementations. The result of this extensive implementation and experimental work is, to the best of our knowledge, the most comprehensive comparative study on Android malware detection methods presented to date. The scientific contributions of this work are summarized as follows:

1. We present a number of factors that negatively affect the accuracy of Android malware detectors. In particular, we consider five conditions that are present in real life but are often ignored when proposing malware detectors: (1) datasets contain a large amount of apps that are almost identical to others; (2) there is not always agreement on what is goodware and what is malware, and some apps lack enough consensus to be considered malicious or benign; (3) there is more goodware than malware; (4) malware authors may resort to evasion attempts using obfuscation techniques; and, (5) malware and goodware evolve over time. Then, we argue that it is imperative to consider all these factors when designing and evaluating Android malware detectors in order to provide realistic performance values.
2. We analyze the performance of state-of-the-art ML-based Android malware detection approaches when the above factors are taken into account. For this purpose, we selected 10 highly influential detectors that make use of static analysis techniques. We show that the outstanding performances provided by the authors of these approaches are unrealistically optimistic due to design and evaluation flaws.
3. We highlight the lack of reproducibility of published work in this area. In this sense, we make the code and datasets used in this comparative work publicly available.
4. We discuss future research lines in Android malware detection aiming to solve the identified design and evaluation flaws.

The structure of this paper is organized as follows. Section 2 reviews the most important works related to the topic of this paper. Section 3 includes basic concepts about Android apps, static analysis and the type of data that can be obtained using this technique. The state-of-the-art detectors based on supervised ML classifiers that are considered in this work are presented in Section 4. Section 5 describes how datasets for malware detection are typically built, their limitations (factors that should be considered when constructing them but are often ignored) and the reproducibility problems that the use of custom datasets entails. In Section 6, we present our experimental setup. Section 7 discusses the limitations of the selected Android malware detectors based on the analysis of the factors identified in Section 5. Section 8 discusses the characteristics that a realistic evaluation framework should take into account, as well as the challenges and future research lines in Android malware detection. We conclude this paper in Section 9.

2. Related work

Android malware detection is a well-studied area in the information security literature. Despite this, only a few experimental studies have focused on analyzing what factors impact the performance of malware detectors, which is the main theme of this paper. Most of them focus their analysis on a small group of detectors with similar characteristics. To date, the most comprehensive study is Pendlebury et al. (2019). This article analyzed two different sources of bias in the evaluations of three detection algorithms. The first, known as spatial bias, comes from the differences between the proportion of samples from each class in the dataset. The second type, temporal bias, is related to the inclusion of future knowledge during model training. Experiments to test for spatial bias concluded that the ratio between classes is determinant in the results reported by the authors. Experiments on the impact of temporal bias demonstrated that models tend to misclassify malware as time passes, whereas the accuracy of goodware remains stable over time. The conclusions drawn from this work are similar to those obtained in two other previous studies, each of which considers only one detector in their experiments (Allix et al., 2016a; Roy et al., 2015).

The use of obfuscation techniques with malware apps to evade detection was studied in Rastogi et al. (2014). The work presented an analysis for ten Android commercial anti-virus products, testing them by applying different obfuscation techniques to malware. The results proved that all the analyzed tools worsened their effective-

[1] https://www.virustotal.com

ness to detect malware with at least one type of obfuscation. This work served to highlight the weaknesses of these commercial solutions. However, given that the details of the detectors are not public, specific conclusions about how obfuscation may affect ML detectors for Android cannot be drawn from this study. Another work assessed the ability of a ML malware detector for Windows to identify packed[2] malware (Aghakhani et al., 2020). An extensive set of experiments was performed to confirm whether packed samples were identified due to the presence of traces left by packers or by the behavior of the sample. The authors concluded that ML detectors relying on static analysis features tend to focus on signs of obfuscation. Thus, putting in question the feasibility of these approaches against Windows malware due to the amount of false positives. Nonetheless, further studies need to be conducted to evaluate if these findings also apply to Android malware detectors.

The presence of duplicates in datasets when designing and evaluating detectors is partially discussed in Surendran (2021). Preliminary experiments are carried out for two ML detectors, one based on the usage of API calls and one using permissions. The authors of that work postulate towards the existence of a relation between duplicates and the obtention of overestimated performances for models. However, additional analysis would be required to confirm this hypothesis, as the implicit reduction of the size of the dataset for extreme duplicate removal configurations (using ample similarity thresholds) may eventually originate similar results.

In summary, previous works have focused on the analysis of some specific evaluation flaws affecting detectors, including: spatial bias (Allix et al., 2015; Pendlebury et al., 2019; Roy et al., 2015), temporal bias (Allix et al., 2015; Pendlebury et al., 2019), the impact of obfuscation (Maiorca et al., 2015; Rastogi et al., 2014) or the influence of duplicates in the data (Surendran, 2021). However, these analyses were conducted over a small number of methods (Allix et al., 2016a; Roy et al., 2015; Surendran, 2021), in some cases, making use of similar feature sets (Pendlebury et al., 2019). Some studies were not deep enough (Surendran, 2021) or were performed exclusively for commercial black-box detectors, so that the details about their detection mechanisms, i.e., their features or whether they are based on signatures or ML algorithms, are not disclosed and results cannot be extrapolated (Maiorca et al., 2015; Rastogi et al., 2014). None of these published works has taken into consideration the bias caused due to the removal from datasets of apps that are neither clearly goodware of malware, or has analyzed the effect of using different thresholds to label an app as malware on many existing detectors. Thus, we believe that a more comprehensive analysis, that includes all the identified factors and a higher number of detectors using different features and ML methods, becomes mandatory in the Android malware detection area. Contrary to published literature, this manuscript not only provides a comparative framework that evidences the lack of realistic proposals and illustrates many extended design and evaluation biases, but also, gives recommendations and identifies future research lines towards the proposal of realistic malware detectors.

3. Background

This section slightly introduces some background necessary to understand the rest of the manuscript. It includes descriptions about the structure of Android apps and the type of information that can be inspected through static analysis of apps to feed ML algorithms.

[2] Developers of malware apps use *packers* to evade detection and analysis. A packer tool compresses and encrypts together data, resources and the code of executable files. These elements are unpacked and executed at run time.

3.1. Android apps

Android apps are distributed in APKs (Android Application Packages). An APK is a ZIP compressed file that contains the resources that are essential for the execution of the app on the system: the manifest data and the compiled code.

3.1.1. Manifest data

The manifest file of an Android app defines a set of properties and components that the app requires from the platform in order to work. This file is in XML format and it is divided into three main blocks or sections:

- The *Application Components* block defines the elements of the app that interact with the OS while the app is running or when a specific action is requested to the OS by the user. These components implement background functionality (services), manage user screens and app interactions (activities), enable app interactions with other OS components or apps (broadcast receivers and intent filters) and the interfaces to share data with other processes (content providers).
- The *Hardware and Software Features* block defines the OS properties and functionalities that the app requires to function. This includes software features such as backup support, user account management, input methods, etc. Hardware features include elements such as the camera, bluetooth transmission, fingerprint sensor, etc. Declaring requested features is useful, for example, to prevent an app from running on a phone that does not fulfill the required specifications.
- The *Permissions* block indicates the features that are required by an app but are protected by the OS. Access to these functionalities must be explicitly granted by the user. By default, Android apps do not have permissions to perform actions that could compromise the OS, user information, or other apps. Thus, permissions are needed to access the microphone, camera, contact list, Internet connection, location, etc.

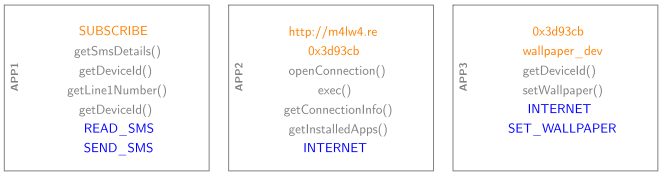
3.1.2. Application code

Android apps are generally developed in Java or Kotlin (Google's preferred language for Android development) and transformed into Dalvik bytecode format during the compilation process. The Dalvik bytecode runs on the Android virtual machine, which serves as a platform-independent environment. Interactions between the hardware components of mobile devices, directly managed by the operating system, and apps, managed by the virtual machine, are performed through API libraries. These APIs provide a common way to access to the hardware capabilities required by the apps. Thus, abstracting the programmer from the particularities of devices.

The Dalvik bytecode of apps is located inside the APK in the classes.dex file. It includes all user-defined classes and functions, as well as constants and variable definitions. External libraries, such as the Android framework, are not part of the content of this file.

3.2. Static analsis of android apps

Static analysis is a software technique that inspects apps to extract their characteristics without the need to execute their code and monitor their behavior at runtime (Li et al., 2017a). By means of static analysis, all execution paths present in the code and all the information in the files of an app can be inspected. This is done by using tools with code interpretation mechanisms that extract understandable structures describing the internal functions of apps, e.g., call graphs, data flows, statistical measures about code structure, etc. Such information is then converted into a common set of explanatory features that will later be processed by ML algorithms.



	wallpaper_dev	http://m4lw4.re	0x3d93cb	SUBSCRIBE	getInstalledApps	openConnection	getSmsDetails	getDeviceId	setWallpaper	getLine1Number	getConnectionInfo	exec	SET_WALLPAPER	INTERNET	SEND_SMS	READ_SMS
APP1	0	0	0	1	0	0	1	1	1	0	1	0	0	0	1	1
APP2	0	1	1	0	1	1	0	0	0	0	0	1	0	1	0	0
APP3	1	0	1	0	0	0	0	0	1	1	0	0	1	1	0	0

Fig. 1. Binary encoding of the strings (in orange), API calls (in black) and permissions (in blue) for three apps.

To perform the static analysis of the content of APK files, tools such as AXMLprinter[3] can be used to make fields and components declared inside the manifest file accessible. Instead, the classes.dex file can be converted to higher level format using decompilers such as *bakSmali*[4]. After that, different features can be obtained, including information about instructions, methods, classes, strings and the usage of API calls (Faruki et al., 2014). It is also possible to build different graph structures representing the code. These include Call Graphs, built following the call instructions (`invoke`) present in the code; and Control Flow Graphs, which are created considering also the jumps in the code caused by conditional and loop statements (`if, switch, for, while`...). In both types of graphs, a node represents a method or a block of instructions that can only be executed sequentially, i.e., a basic block; and the edges represent the execution flow between nodes (Li et al., 2017a).

After performing static analysis, the data obtained from APKs is mapped into feature vectors that represent the apps in a structured way, suitable for processing by ML algorithms. Fig. 1 depicts the binary mapping as applied to represent three apps by means of their strings, API calls and permissions. It uses the values 1 or 0 to denote whether a feature is present in an app's code or not. Other mappings are also possible, for example, frequency encoding accounts for the number of times a feature is present in the app code.

4. Android malware detectors based on supervised classifiers

Supervised classification is a popular ML task in which the objective is to learn a mapping or *classifier* $\hat{H} : \mathcal{X} \rightarrow \mathcal{C}$, where $\mathcal{X}$ is a space of features that describes the samples (the input), and $\mathcal{C}$ is the space of class labels (the output). To do so, the ML algorithm is fed with a set of labeled samples $D = \{(\boldsymbol{x^1}, c^1), \ldots, (\boldsymbol{x^n}, c^n)\}$ called the training set. In the context of Android malware detection using ML classifiers, the training set consists of a set of labeled apps. Each app or sample is described as a vector of t features $\boldsymbol{x^i} = (x_1, x_2, \ldots, x_t)$ which, in the context of this work, are computed through the static analysis of its APK file. The binary class label c^i takes a value of 0 for goodware apps and a value of 1 for malware apps. Once the classifier is trained, given the feature vector of a new app $\boldsymbol{x^k}$, it will return its predicted class label $\hat{c}^k$ (0 for goodware and 1 for malware).

As mentioned, for the purpose of this work we selected 10 malware detection methods for Android. Eight of them are selected because they have been published in top-tier journals and represent the most important papers in this area of study, in terms of relevance, according to IEEExplore, Scopus and Google Scholar. We have also added two additional proposals, which have been considered in other experimental comparative works, namely the Drebin (Arp et al., 2014) and BasicBlocks (Allix et al., 2016a). All of these are *misuse*-based detectors, using different supervised ML classification algorithms and with different sets of features extracted from the apps. In addition, given the large number of features they obtain from APKs, some approaches apply dimensionality reduction algorithms (Molina-Coronado et al., 2020).

Table 1 outlines the key aspects of these detectors. As can be seen, they were published between 2014 and 2019. Due to the lack of standard datasets, an aspect which is studied in depth in the next section, they all used custom collections of apps to perform their experiments. In many cases, for these proposals, sample selection, class ratios or labeling criteria not only vary, but are not properly documented also. This makes it difficult to reproduce the experiments, compare proposals or measure their contribution level. Furthermore, none of their authors, excepting those of HMMDetector and MaMaDroid, have made their code publicly available, hindering the possibility to re-run the detectors, and complicating their use for comparison purposes. Because of these, a direct comparison of the performances reported in the corresponding articles does not provide useful information.

Given the unavailability of the code of most of the detectors, we have had to re-implement them in order to perform the experimental analysis contained in this work. We tried to reimplement every proposal in the best possible way, so that we can offer fair and accurate comparisons. However, this was a complex task, mainly due to the lack of details concerning crucial aspects such as parameter values, the feature extraction and training processes of the classifier, etc. The most problematic approaches in this sense have been MultimodalDL and PermPair. Unfortunately, we were unable to implement MultiModalDL (Kim et al., 2018), due to its complexity and the omission of information regarding feature computations (number of centroids, thresholds for similarity computation). As for PermPair (Arora et al., 2019), we implemented the detector as indicated in the original publication. However, the results obtained by us are far from those originally reported by the authors. For these reasons, we omit these works from subsequent analysis.

We have implemented the remaining eight malware detectors listed in Table 1 in Python language.[5] Note that we have carried out the necessary static analysis tasks on APKs to obtain the feature sets specific to each detector, as explained in Section 3. For this purpose, we used the *Androguard* framework (Desnos et al., 2018), a widely-used static analysis and reverse engineering tool for Android APK files. For dimensionality reduction, ML algorithms and the assessment of the performance of detectors, we employed well-established libraries such as *scikit-learn* (Pedregosa et al., 2011) and *numpy* (Harris et al., 2020). Our goal is not only to perform a comparative analysis between Android malware detectors, but also to contribute to the reproducibility and progress in the

[3] https://www.github.com/tracer0tong/axmlprinter

[4] https://www.github.com/JesusFreke/smali

[5] In the case of DroidDetector, we were able to implement the detector assuming that feature selection was used to identify the set of most relevant API calls used by the algorithm.

Table 1
Android malware detection methods included in this analysis. They are the most relevant works according to the literature.

Method	Pub. Year	Dataset (#samples)	APK Features	Encoding	Dim. Reduction	ML Algorithm	Reported Performance
AndroDialisys Feizollah et al. (2017)	2017	Google Play[a] (1 846) Drebin (5 560)	Permissions Intent Filters	Binary	-	Bayesian Network	TPR: 0.955 FPR: 0.044
BasicBlocks Allix et al. (2016a)	2016	Google Play[a] (52 000) Android Genome (1 260)	Basic Blocks	Binary	Mutual Info	Random Forest	TPR: 0.91 Precision: 0.94
Drebin Arp et al. (2014)	2014	Benign[a] (123 453) Drebin (5 560)	Permissions App. Components Hw. Components Intents Strings API calls	Binary	-	SVM	TPR: 0.939 FPR: 0.01
DroidDet Zhu et al. (2018)	2018	Google Play[a] (1 065) VirusShare[a] (1 065)	Permissions Intents API calls	Binary	tf-idf rank	Rotation Forest	TPR: 0.884 Precision: 0.886
DroidDetector Yuan et al. (2016)	2016	Google Play[a] (20 000) Android Genome (1 260) Contagio[a] (500)	Permissions API calls	Binary	Mutual Info[b]	DBN	TPR: 0.981 FPR: 0.201
HMMDetector Canfora et al. (2016)	2016	Google Play[a] (5 560) Drebin (5 560)	Opcodes	Sequence	HMM	Random Forest	TPR: 0.968Precision: 0.96
ICCDetector Xu et al. (2016a)	2016	Google Play[a] (12 026) Drebin[a] (5 264)	Intents App. Components	Binary and frequencies	Mutual Info	SVM	TPR: 0.931 FPR: 0.006
MaMaDroid Onwuzurike et al. (2019)	2019	Google Play[a] (8 447) Drebin (5 560) VirusShare[a] (29 933)	API call graph	Frequencies	-	Random Forest	TPR: 0.97 Precision: 0.95
MultimodalDL Kim et al. (2018)	2018	Google Play[a] (20 000) Android Genome (1 260) VirusShare[a] (20 000)	App. Components Intents Hw. Components Permissions Opcodes Strings API calls	Binary	-	DL	TPR: 0.99 Precision: 0.98
PermPair Arora et al. (2019)	2019	Benign[a] (6 993) Android Genome (1 264) Drebin[a] (2 764) Contagio[a] (250) Koodous[a] (2 975) PwnZen[a] (300)	Permissions	Frequencies	Sequential removal	Nearest Neighbors	TPR: 0.951 FPR: 0.042

[a] Unspecified set or labeling criteria from this source.
[b] In the original paper, authors use a filtered set of API calls without stating the criterion used to extract this set. As this information is unavailable, we decided to use Mutual Information to select the most important API calls based on the training data.

area by releasing our programs and the instructions for their use in our GitLab repository.[6]

5. Datasets for android malware detection. Drawbacks and reproducibility issues

When building a supervised classifier, the dataset and the procedures used for training and evaluation are key factors in order to develop robust and well-performing models (Hastie et al., 2009). Consequently, for ML-based malware detection, the collection of apps and the process for their generation are particularly important, both in terms of applicability to real-world scenarios and reproducibility. In this section, we identify and discuss a set of factors that must be considered when creating datasets. We then describe the datasets most commonly used by the Android malware detection research community, pointing out their main drawbacks from the point of view of the identified factors.

5.1. Factors under analysis

We have identified five factors that have a major impact on the performance of ML based malware detectors for Android, and that should be considered when creating the datasets used to train and evaluate them. It should be noted that, in our opinion, these factors are very important, but this does not imply that they are the only influential aspects in the design and evaluation of malware detectors based on supervised classification.

5.1.1. REDUNDANCY

The purpose of Android malware detectors is to identify every type of malware regardless of their level of incidence. This includes a wide range of samples pertaining to different malware families and subfamilies. Typically, malware samples within a family or subfamily exhibit analogous code and data structures to perform the same malicious activities (Wei et al., 2017). This characteristic of malware results in datasets with groups of samples that are very similar from the point of view of detectors. This means that, within these groups, samples tend to be represented using identical feature sets, resulting in redundancies in the data fed to ML models. On the one hand, redundant samples in the training set cause bias in ML algorithms because decisions towards groups with many representatives have a great impact in the accuracy of the model (Hastie et al., 2009). This makes models to ignore non-redundant samples, since predictions for these instances yield little improvement in accuracy. Resulting, thus, on detectors with a limited ability to identify uncommon (less represented) forms of malware. On the other hand, the presence of duplicates in the evaluation set leads to inflated or poor performances, depending on whether or not these large groups of similar apps are correctly classified. Even

[6] https://gitlab.com/serralba/androidmaldet_comparative.

if the presence of duplicates or very similar apps has an important effect on the results obtained by a detector, to the best of our knowledge, only preliminary work has considered it as an issue (Surendran, 2021). As a matter of fact, none of the detectors considered in our comparative analysis were assessed taking into account redundancies in the data as an major source of bias.

5.1.2. LABELING-GREYWARE

One of the main problems when building datasets for Android malware detectors based on supervised classification lies in the need to have labeled samples. The true nature of an app is not known in many cases. This implies that apps need to be analyzed in order to assign them a label concerning their maliciousness. Ideally, labeling should be carried out manually, by experts, to guarantee the highest number of error-free labels. However, using human annotators to perform this task is costly, both in terms of time and resources (Li et al., 2017b) and it is not error-free either. Accordingly, researchers usually rely upon automatic procedures to label their customized datasets.

In the simplest labeling approach, apps are labeled according to the source they were obtained from (Arora et al., 2019), e.g., those downloaded from trusted repositories (such as Google Play) are goodware, whereas those from repositories such as VirusShare are labeled as malware. This procedure relies on the analysis, either automatic or manual, performed by the managers of these repositories. In other cases, apps are scanned using a collection of antivirus programs and labeled depending on the number of positive (malware) alerts (Souri and Hosseini, 2018). In this context, VirusTotal is a tool that allows users to upload files, including APKs, and scan them using a collection of antivirus engines. VirusTotal results, based on the number of positive alerts raised by a file (we refer to this number as VTD, from VirusTotal Detections), are widely used as the criterion to label samples. To this end, a common procedure makes use of thresholds to establish the level of consensus required to label an APK as malware or goodware (Salem et al., 2019).

Leveraging on the VTD leads authors to decide what threshold is adequate for malware and goodware, so it is common to find disparities in the literature. For example, (Roy et al., 2015) set a threshold of VTD$\geq$10 to flag an app as malware, while in Pendlebury et al. (2019), authors set this threshold at a much lower value of 4. In both cases, the condition for labeling an app as goodware was set to VTD = 0. The choice of thresholds not only means that the malware or goodware definitions vary among articles, but also influences the amount of apps that fall between these categories. These apps, which we refer to as greyware, lack enough consensus by antivirus programs to be considered as goodware or malware with guarantees. Because of that, many researchers discard these apps when training their detectors. However, greyware is an important part of the Android ecosystem (Google, 2019a) and will appear whenever the detector is deployed in a real environment. Thus, discarding them at training time will hinder the effectiveness of a detector once deployed. Indeed, even if common thresholds were applied to label the data, the VTD value provided by VirusTotal changes over time (Zhu et al., 2020), for example, due to engine updates aimed at improving detection capability, or as engines are added or removed from the platform. Consequently, disparities may occur between models validated with exactly the same collection of apps but labeled at different times (Salem et al., 2019).

5.1.3. IMBALANCE

The third factor we consider is related to the ratio of malware and goodware in the dataset, especially in the data used for training. In real life, most apps are innocuous in the security aspect, with the actual proportion of malware being about 10% of the total number of Android apps (Pendlebury et al., 2019), but with this ratio being highly dependent on the particular market from which apps are downloaded (Google, 2019b; Lindorfer et al., 2014). As can be seen in Table 1, researchers have trained their detectors following their own criterion and assuming different class proportions. However, the choice of the class ratio is important when training ML models, since classical ML classifiers tend to be biased towards the majority class in highly unbalanced scenarios. Therefore, ignoring class imbalance can lead to a false perception about the true performance of detectors under real working conditions (Pendlebury et al., 2019).

5.1.4. EVASION

Some malware authors are aware that their apps will be examined by malware detectors, so they try to bypass detection by using different evasion techniques. As technology evolves, different threats may appear and authors should be aware that their systems will be the target of attacks. Thus, a proper evaluation of the robustness of malware detectors against such threats should be taken into consideration. Unfortunately, none of the proposals included in this comparative work have considered the effects of evasion techniques, such as obfuscation, in their evaluations.

5.1.5. EVOLUTION

The last factor we highlight is related to the evolution of malware and goodware over time. This implies that the behavior of apps rarely remains static for a long time. Thus, the characteristics of newer or mutated apps may differ from those obtained from apps observed earlier, during the training of a detector. Despite that, most authors design and/or evaluate their ML detectors on the assumption that future malware and goodware will remain similar to that used at design time (Pendlebury et al., 2019; Roy et al., 2015).

5.2. Available android datasets and their drawbacks

Android datasets that are used for building supervised classifiers consist of collections of apps and their associated labels, which indicate whether an app is malware or goodware. We have searched in the literature for datasets of Android APKs designed for research on misuse-based Android malware detectors, and have found five popular ones. Their characteristics have been summarized in Table 2, taking into account all the factors identified in the previous section.

In two of the reported datasets (Drebin and CICAndMal2017), the labels of the samples were obtained by setting some threshold over the VTD. For example, in Drebin, an app is tagged as malware when its VTD$\geq$2, for a subset of eight antivirus engines from VirusTotal that are selected as reliable by the authors. Only the Android Genome dataset was built based on manual labeling. A combination of both labeling approaches was used in the AMD collection: automatic labeling was first carried out using VirusTotal to filter and cluster apps into malware families, and then a small subset from each family was manually verified. Finally, note that AndroZoo does not provide labels, supplying VTD values instead –so it is up to the user how to use this information for labeling.

To properly train detectors based on ML classifiers, both malware and goodware samples are needed. Ideally, greyware should also be included. Nonetheless, Drebin, Android Genome and AMD comprise exclusively malware samples and only AndroZoo allows samples to be labeled as greyware. Another drawback of these datasets is related to obfuscated malware. In this sense, the authors neither identify, or explicitly include, obfuscated versions of malware, which makes it very difficult to analyze the possible effects of evasion attempts on the performance of detectors. Some-

Table 2
Characteristics of popular Android Malware Datasets. The "?" symbol means that no information about this characteristic is reported.

Dataset	Time period	Labeling method (Type of labels)	#samples	Obfuscated samples	Redundant samples	Timestamps
Android Genome Zhou and Jiang (2012)	2010–2011	Manual (Binary)	1 260 malware	?	?	✗
Drebin Arp et al. (2014)	2010–2012	VirusTotal (Binary)	5 560 malware	?	?	✗
AMD Wei et al. (2017)	2010–2016	Hybrid (Binary)	24 553 malware	?	?	✗
CICAndMal2017 Lashkari et al. (2018)	2014–2017	VirusTotal (Binary)	426 malware 5 065 goodware	?	?	✗
AndroZoo Allix et al. (2016b)[a]	2011-	VirusTotal (VTD)	13,045,285 mixed	?	?	✓

[a] This dataset is continuously growing

thing similar occurs for redundant samples, as none of the papers describing the datasets provide information about their presence.

Finally, as can be seen in Table 2, most of these datasets were created more than five years ago and may contain old-fashioned malware. The most recent dataset is CICAndMal2017, which comprises a small set of apps released between 2014 and 2017. This small number of instances may not be enough for training and evaluating anti-malware methods based on some ML algorithms (Raudys et al., 1991). In addition, in most cases the samples included in these datasets cover only a small period of time or their release date is not available. These facts complicate the elaboration of proper analysis about the evolution of the characteristics of apps.

The limitations of these datasets have led researchers to build custom datasets, in a similar way as AndroZoo was created, i.e., by combining APKs downloaded from various sources or repositories. AndroZoo constitutes the most important public source of Android apps for researchers (Allix et al., 2016b). Sources of AndroZoo include app marketplaces (such as Google Play), malware datasets, torrents and different malware repositories[7] Since 2011, these sources are continuously tracked to keep the collection up to date. Additionally to the APKs, AndroZoo provides a file with information about the apps contained in the dataset. The contents of this file, which is updated daily, can be used to filter the samples to be downloaded according to different criteria, such as the market from which an APK was obtained, the SHA and the date of the APK, and the VTD value.

From the above discussion, it is clear that the decisions made by the authors during the construction of custom datasets make them unique, and that particularities of these datasets influence the performance of detectors. Moreover, performance is not only dependent on the data used, but also on the design of the experiments using these data. In our opinion, and as mentioned in the previous section, the use of different datasets and the consideration of different guidelines in the design of the experiments, makes the comparison of the metrics reported for these proposals meaningless, entails an obstacle to the reproducibility and seriously impairs progress on this relevant topic.

6. Experimental setup

In this section we present the experimental setup devised for this work. We describe the datasets used, the process to build and evaluate the Android malware detectors and the evaluation metrics considered to measure their performance.

6.1. Master dataset

Our starting point is a "master" dataset that allows us to derive datasets useful to evaluate on the selected detectors every factor presented in Section 5.1, i.e., REDUNDANCY, LABELING-GREYWARE, IMBALANCE, EVASION and EVOLUTION.

To build our master dataset we selected apps from AndroZoo, taking into account their origin, the reported VTD and the date. We use apps from Google Play, AppChina and VirusShare. The date of the apps was used to select 100 monthly samples of each class (malware, goodware and greyware) during the period starting from January 2012 to December 2019. As we mentioned earlier, there is a lack of ground truth in the area, and automated labeling methods based on VirusTotal have become commonplace. Given the lack of standard criteria in the literature to interpret the results obtained from VirusTotal (Salem et al., 2019), in this paper we resort to a simplified but operational automated labeling method: the use of VTD values and the application of a set of pre-defined thresholds. With this method we have a practical definition of what is goodware, what is greyware and what is malware that allows us to fairly compare detectors and to show different sources of experimental bias. In particular, we used the median of the thresholds used in two previous works (Pendlebury et al., 2019; Roy et al., 2015) to label the malware, i.e., a VTD$\geq$7. Goodware samples were considered as those with VTD=0, whereas samples with a 1$\leq$VTD$\leq$6 rating were labeled as greyware. Moreover, we restricted our analysis to samples discovered at most in 2019 to ensure that malware signatures for samples in our dataset are well stabilized. Such criteria are in concordance with what is recommended in Zhu et al. (2020) to obtain reliable labels from VirusTotal.

In total, our master dataset consists of 28,800 samples. The complete list of APKs and the instructions to download this dataset, along with our code, are available in our GitLab repository. In contrast to the datasets introduced in Section 5.2, this dataset is more comprehensive as: (1) it considers greyware; (2) it provides a sufficiently large number of goodware, malware and greyware samples to properly train and test ML detectors; (3) it considers a larger time period and the organization of samples by months allows us to split our dataset according to the age of the apps. Additionally, compared to the datasets used for training the detectors included in this analysis (see Table 1), the procedure followed to build our master dataset and its derived datasets is more systematic, rigorous and transparent.

6.2. Model training and assessment process

For model construction and evaluation, the dataset is divided into training and test partitions. In order to obtain unbiased results, the test partition is always kept as a completely separate set and is never used for training nor for feature engineering processes (extraction, preprocessing or dimensionality reduction). The model parameters are selected using standard k-fold cross validation (with $k = 5$) within the training set and following a grid search approach. In scenarios where EVOLUTION is considered because the distributions of the data are assumed to change over time, time-aware k-folds are used (Arlot and Celisse, 2010), i.e., all the apps of the evaluation sets are more recent than those for training. In all cases, folds are created maintaining the original ra-

[7] For example: ContagioDump (https://www.contagiodump.blogspot.com/), Koodous (https://www.koodous.com), VirusShare (https://www.virusshare.com/).

Table 3
Evaluation metrics used in this paper.

$TPR = \frac{TP}{P}$	$FPR = \frac{FP}{N}$	$Precision = \frac{TP}{TP+FP}$
$F1 = 2 * \frac{TPR * Precision}{TPR + Precision}$		$A_{mean} = \frac{TPR + 1 - FPR}{2}$
$kappa = \frac{P_o - P_c}{1 - P_c}$		$P_o = \frac{TP+TN}{P+N}$ $P_c = \sum_k p_k * \hat{p}_k$

tio between the classes in the given experimental scenario. This is a common process in the ML literature for estimating the generalization error of the final model (Hastie et al., 2009).

6.3. Evaluation metrics

For this work, we considered a set of evaluation metrics (see Table 3) which are common in the ML and computer security literature (Molina-Coronado et al., 2020). Note that some of these metrics, such as the TPR, the FPR or precision, offer complementary information and should be used together to fully understand the performance of a system. These metrics make use of: true positives (TP), which indicate the number of correct positive predictions; false positives (FP), that account for the number of incorrectly predicted negative elements; and P and N, which make reference to the number of positive and negative elements in the data. Conversely, summary metrics such as A_{mean}, F1 score or kappa statistic, quantify the performance of a system using one measure, but lack explainability and do not provide insights regarding the causes of the good or bad performance. F1 represents the harmonic mean between precision and TPR, A_{mean} is the arithmetic mean of the accuracies for the positive and negative classes, and the kappa statistic quantifies the level of correlation between the predictions made by a classifier and the actual labels in the data (P_o being the accuracy of the detector, P_c being the weighted sum of the predictions, p_k being the actual proportion of samples of the class k in the data and $\hat{p}_k$ being the proportion of samples predicted as pertaining to class k). Unlike the F1 score (as it does not consider True Negatives), the kappa and A_{mean} metrics are especially useful under unbalanced problems.

7. Comparative analysis

In this section we run a complete set of experiments, designing specific scenarios to analyze the effect of the factors presented in Section 5.1. First, a basic scenario is used as the departing point for subsequent experiments. In all scenarios, the selected detectors are compared in equal conditions.

Departing from the master dataset, two different datasets have been extracted, namely balanced and unbalanced (see Table 4). For the balanced case, it is assumed that malware and goodware are equally probable. Thus, from the period 2012–2015, ∼70% of all the goodware and ∼70% of all the malware are uniformly sampled on a monthly basis and used for training, keeping the remaining 30% of the samples for testing purposes. On the contrary, under the unbalanced configuration, malware is assumed to be less frequent than goodware and, consequently, the malware is downsampled to become ∼10% of the goodware. This process is performed by randomly sampling malware on a monthly basis according to a Gaussian distribution with parameters $N(0.1, 0.02)$.

7.1. Baseline scenario

Before analyzing the scenarios related to each factor, we test detectors under the most basic and unrealistic assumptions, i.e., discarding greyware, considering a balanced ratio between the classes, omitting evasion attempts and ignoring the evolution of the apps. The aim of this scenario is to mimic the (favorable) conditions that are commonly assumed in the literature and try to reproduce the results obtained in the original papers.

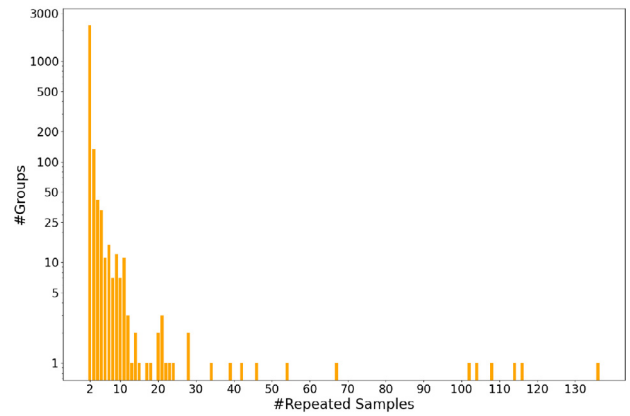


Fig. 2. Number of malware groups per group size. Note the logarithmic scale in the Y axis.

As shown in Table 5, half of the detectors showed promising detection performances in this scenario, achieving detection rates (TPR) over 0.8 and moderately low false positives of about 0.1. The best detector under this scenario in terms of summary metrics (0.91 for kappa), despite being one of the first works in the area, was Drebin, with TPR and FPR values of 0.95 and 0.04, respectively. Another two well-performing detectors were DroidDet and MaMaDroid, with similar kappa figures of 0.84, despite using different features and ML algorithms. The good results reported for some methods contrast with those obtained for DroidDetector, HMMDetector and AndroDialysis. Given the high number of false positives of these detectors (above 20%), one could conclude that these models are inappropriate even under optimistic working conditions.

7.2. Redundancy scenario

This scenario aims to study the impact of the presence of very similar samples in Android datasets. To this end, we filter out malware and goodware apps from our initial balanced dataset. This is done by computing intra-class similarities between samples and then applying the algorithm proposed in Surendran (2021). The algorithm works by randomly selecting one sample at a time and removing the samples lying in its neighborhood, according to an ϵ radius threshold.

For our experiments, we use call frequencies of APIs as the representation of apps and the Euclidean distance to compute the degree of similarity between samples. The intuition behind the consideration of this representation lies in that API call frequencies make reference to both the set of API calls that describe the actions carried out by apps, and the prevalence of these calls in the code. Thus, we assume that when the Euclidean distance for two slightly different apps is below or equal to ϵ, both apps perform identical actions, e.g., they are variations of the same app.

We perform the filtering process using a redundancy tolerance value ϵ equal to 0, which means that we consider as duplicates two apps with *exactly* the same API call frequencies. As a result of this process, a dataset without exact duplicates is obtained. A summary of the characteristics of this dataset are presented in Table 6. As can be seen, the size of the malware subset is reduced substantially (almost half of the malware was filtered out), whereas for goodware the number of removed duplicates is considerably smaller. Details about the groups found for malware during the deduplication process are shown in Fig. 2. It depicts the number

Table 4
Composition of Balanced and Unbalanced datasets derived from the 2012–2015 period of the Master Dataset.

	Training			Testing		
	%malware (#samples)	%goodware (#samples)	ratio	%malware (#samples)	%goodware (#samples)	ratio
Balanced	70% (3360)	70% (3360)	1:1	30% (1440)	30% (1440)	1:1
Unbalanced	∼7% (∼336)	70% (3360)	∼1:10	∼3% (∼144)	30% (1440)	∼1:10

Table 5
Performance of detectors for the baseline scenario.

Method	TPR	FPR	Precision	F1	A_{mean}	Kappa
AndroDialysis	0.848	0.291	0.744	0.792	0.778	0.556
BasicBlocks	0.823	0.097	0.894	0.857	0.863	0.726
Drebin	0.953	0.043	0.956	0.955	0.955	0.910
DroidDet	0.936	0.088	0.913	0.925	0.924	0.848
DroidDetector	0.472	0.335	0.585	0.523	0.568	0.137
HMMDetector	0.824	0.560	0.595	0.691	0.631	0.263
ICCDetector	0.800	0.093	0.895	0.845	0.853	0.706
MaMaDroid	0.930	0.085	0.915	0.923	0.922	0.845

Table 6
Composition of the full dataset used for the filtered scenario (data from 2012 to 2015).

Dataset	#malware	#goodware
Original	4800	4800
Filtered	2588	4291

Table 7
Performance of detectors trained using the filtered dataset and evaluated with the baseline test set.

Method	TPR	FPR	Precision	F1	A_{mean}	Kappa
AndroDialysis	0.904	0.365	0.712	0.796	0.769	0.538
BasicBlocks	0.893	0.044	0.952	0.922	0.924	0.849
Drebin	0.933	0.031	0.966	0.949	0.950	0.901
DroidDet	0.933	0.056	0.942	0.937	0.938	0.876
DroidDetector	0.654	0.403	0.618	0.635	0.625	0.250
HMMDetector	0.796	0.488	0.619	0.697	0.653	0.307
ICCDetector	0.800	0.053	0.937	0.863	0.873	0.747
MaMaDroid	0.914	0.046	0.951	0.932	0.934	0.868

of groups found int the malware dataset arranged by group size. As can be seen, our filtered dataset is heterogeneous in terms of unique malware behaviors. More specifically, 2285 samples only contain a duplicate, which represent 88% of the filtered malware. Groups with a small number of duplicates represent the majority of the data, with 70% of the original malware containing less than 10 identical samples. Large groups are also present. In this regard, about 2% of the resultant malware groups have more than 10 duplicates.

In order to confirm the influence of the presence of very similar apps in the generalization ability of ML models, the results for detectors trained with filtered data[8] and evaluated with the unfiltered test set used for the baseline scenario are shown in Table 7. As can be seen, almost all approaches improve their performances, with similar TPR and lower FPR values, compared to the results for detectors trained with the unfiltered training set (see Table 5). Among the most benefited methods in terms of performance are ICCDetector and BasicBlocks, with improvements of 5% and 16% in their kappa values, respectively. The results observed for this scenario support the hypothesis that duplicate samples in the training set result in biased models, since accurate predictions for these large groups at training time result in higher accuracy, and the models are less prone to pay attention to minority groups. Thus, removing duplicates from training is translated into models that are able to generalize better to different types of instances.

[8] As in the baseline scenario, a balanced ratio between goodware and malware is used for training. This is achieved by randomly downsampling the majority class (goodware).

Table 8
Performance of detectors trained and evaluated with filtered sets.

Method	TPR	FPR	Precision	F1	A_{mean}	Kappa
AndroDialysis	0.913	0.362	0.716	0.802	0.775	0.551
BasicBlocks	0.880	0.065	0.930	0.904	0.907	0.814
Drebin	0.925	0.056	0.942	0.933	0.934	0.868
DroidDet	0.926	0.073	0.926	0.926	0.926	0.853
DroidDetector	0.728	0.395	0.647	0.685	0.666	0.332
HMMDetector	0.774	0.545	0.586	0.667	0.614	0.229
ICCDetector	0.797	0.070	0.918	0.853	0.863	0.726
MaMaDroid	0.900	0.072	0.925	0.913	0.914	0.828

The previous experiment was carried out with an unfiltered test set. However, to evidence the impact that the presence of duplicates in the testing set has on the reported performances, we perform evaluations using a filtered test set. Table 8, depicts the performance for detectors both trained and evaluated using data without duplicates. Among all the methods, the best scoring are Drebin and DroidDet with kappa values of 0.86 and 0.85, respectively. However, as expected, most detectors report lower detection performances in this scenario than in the previous experiments (see Tables 7 and 5). With duplicates in the test set, correctly identifying bigger groups has a significant positive effect on the measured performance, whereas erroneously classifying these groups penalizes it. This effect disappears when duplicates are removed and only one representative per group is left. Therefore, the measured performance is a better indicator of generalization ability of detectors.

7.3. Labeling-greyware scenarios

As previously mentioned, manual, exhaustive and error-free labeling is not affordable. For this reason, the community has agreed to use tools such as VirusTotal to automatically label apps. In this sense, using thresholds on the VTD value is the most common class separation criteria. However, different interpretations and thresholds have been used, which impacts on the obtained datasets. Two interesting analyses arise here: (1) how the selection of the threshold affects performance and, (2) what is the behavior of detectors when they examine apps that were discarded during labeling.

First, we evaluate how the selection of different criteria for labeling datasets affects detectors and thus, the reported performance. Specifically, we want to analyze whether the uncertainty and difficulty of the problem is greatly reduced when higher VTD values are used for labeling malware. To do so, models are trained similarly to the baseline scenario, i.e., using a balanced ratio between both classes with samples selected from 2012 to 2015, but varying the threshold used for labeling.

Results for this experiment are shown in Table 9. As can be seen, the TPR of models is directly correlated with the VTD used for labeling the dataset i.e., the higher the VTD, the higher the TPR

Table 9
Performance of detectors trained with data labeled using different thresholds over the VTD for the malware. TPR stands for the True Positive Ratio and FPR is the False Positive Ratio.

	VTD>=1		VTD>=2		VTD>=3		VTD>=4		VTD>=5		VTD>=6	
Method	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR	TPR	FPR
AndroDyalisis	0.547	0.258	0.622	0.275	0.549	0.183	0.692	0.305	0.798	0.369	0.861	0.298
BasicBlocks	0.731	0.184	0.869	0.112	0.885	0.107	0.904	0.134	0.909	0.090	0.916	0.083
Drebin	0.731	0.227	0.860	0.113	0.901	0.107	0.914	0.131	0.935	0.081	0.923	0.062
DroidDet	0.752	0.236	0.873	0.153	0.901	0.152	0.904	0.174	0.935	0.154	0.930	0.083
DroidDetector	0.526	0.384	0.826	0.625	0.937	0.813	0.865	0.757	0.751	0.467	0.763	0.506
HMMDetector	0.759	0.600	0.775	0.588	0.804	0.589	0.794	0.536	0.733	0.506	0.784	0.562
ICCDetector	0.612	0.180	0.732	0.120	0.780	0.096	0.797	0.156	0.832	0.141	0.833	0.055
MaMaDroid	0.756	0.199	0.879	0.136	0.923	0.159	0.923	0.187	0.901	0.145	0.895	0.076

Table 10
Decisions made by baseline detectors when dealing with greyware samples from the period 2012–2015, for different VTD values. G stands for the ratio of identifications as goodware, while M is the ratio of identifications as malware.

	VTD=1		VTD=2		VTD=3		VTD=4		VTD=5		VTD=6		1<=VTD<=6	
Method	G	M	G	M	G	M	G	M	G	M	G	M	G	M
AndroDyalisis	0.504	0.496	0.356	0.644	0.272	0.728	0.307	0.693	0.275	0.725	0.200	0.800	0.319	0.681
BasicBlocks	0.675	0.325	0.452	0.548	0.394	0.606	0.333	0.667	0.248	0.752	0.248	0.752	0.392	0.608
Drebin	0.696	0.304	0.460	0.540	0.379	0.621	0.323	0.677	0.251	0.749	0.148	0.852	0.376	0.624
DroidDet	0.700	0.300	0.437	0.563	0.336	0.664	0.297	0.703	0.221	0.779	0.159	0.841	0.358	0.642
DroidDetector	0.629	0.371	0.691	0.309	0.607	0.393	0.659	0.341	0.607	0.393	0.536	0.464	0.622	0.379
HMMDetector	0.415	0.585	0.296	0.704	0.311	0.689	0.259	0.741	0.255	0.745	0.163	0.837	0.283	0.717
ICCDetector	0.683	0.317	0.460	0.540	0.386	0.614	0.381	0.619	0.295	0.705	0.240	0.760	0.408	0.593
MaMaDroid	0.677	0.323	0.423	0.577	0.366	0.634	0.313	0.687	0.217	0.783	0.137	0.863	0.356	0.645

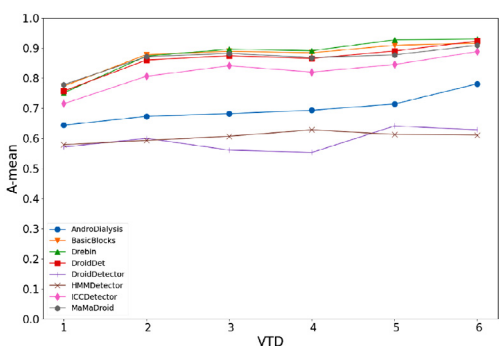


Fig. 3. A_{mean} values for different VTD labeling thresholds.

Table 11
Label swaps for goodware (G), greyware (X) and malware (M); between annotations of samples in the master dataset using information contained in AndroZoo (rows) and VirusTotal reanalysis reports (columns).

		VirusTotal reanalysis		
		G	X	M
AndroZoo analysis	**G**	8 563	954	83
	X	2 613	4 284	2 703
	M	17	663	8 920

of the detector. The contrary is true for the FPR, since it follows a reverse trend with respect to the VTD. This confirms that the malware detection problem becomes easier whenever the separation between both classes, in terms of the VTD, is enlarged. Fig. 3 exhibits this effect for the values of A_{mean} obtained for the different detectors.

The second scenario is devoted to show how detectors behave when facing greyware. Specifically, we want to analyze whether the uncertainty and difficulty of the problem increases when adding greyware and, thus, the omission of these apps results on the oversimplification of the problem and leads to unrealistic and unfair evaluations. To do so, models are trained using the balanced dataset and tested exclusively on greyware samples from the period between 2012 and 2015.

We have compiled the responses of the detectors when classifying greyware samples in Table 10. G indicates how many input samples are identified as goodware, whereas M refers to how many samples are classified as malware. The total results are in the right column of the table. The other columns show the partial results for the input samples grouped by their VTD scores. As can be seen, on average 35% of the total samples are considered goodware by detectors. As expected, the results show certain correlation between the VTD and the decisions made by the detectors: given an app, the higher its VTD is, the more likely the detectors are to classify it as malware. We can observe a high uncertainty for samples with lower VTD values, explaining why authors opt to discard greyware from their experiments. As a result, the problem solved by detectors is simplified. Thus, providing artificially boosted performance results and hiding an effect that will appear in real-working conditions.

In addition, we also analyze how the VTD value and thus, the labels, change over time. For the labeling of our dataset we used the VTD scores provided by AndroZoo. We completely reanalyzed the apps in our dataset with VirusTotal and computed the confusion matrix to represent the label swaps between the two analyses. As can be seen in Table 11, most swaps occur for the greyware class. In this sense, more than half of the apps change their label from greyware to goodware (27.2%) or malware (28.1%). Also, 9.9% of the goodware and 6.9% of the malware apps fall into the category of greyware after being reanalyzed. These changes illustrate the downsides and limitations of using the VTD score for labeling. On the one hand, the need of providing the labels of samples when releasing datasets to avoid class swaps caused by the reanalysis of apps and to guarantee reproducibility. On the other hand, the need to reconsider greyware as part of the datasets, as 44% of the apps remain in this category even after being reanalyzed.

Table 12
Performance of detectors trained with 1:10 malware/goodware ratio.

Method	TPR	FPR	Precision	F1	A_{mean}	Kappa
AndroDialysis	0.373	0.032	0.543	0.442	0.670	0.396
BasicBlocks	0.460	0.017	0.734	0.565	0.721	0.531
Drebin	0.740	0.017	0.816	0.776	0.861	0.754
DroidDet	0.686	0.013	0.837	0.754	0.836	0.731
DroidDetector	0.266	0.162	0.145	0.188	0.552	0.076
HMMDetector	0.106	0.010	0.516	0.176	0.548	0.149
ICCDetector	0.520	0.013	0.795	0.629	0.753	0.599
MaMaDroid	0.513	0.008	0.865	0.644	0.752	0.617

7.4. Imbalance scenario

According to previous literature, around 10% of apps are actually malware (Pendlebury et al., 2019). Similarly, in this scenario we assume that there will be a strong imbalance between malware and goodware. In order to analyze the behavior of the different detectors in this context, the unbalanced dataset is used. It is worth noting that making the test set balanced (or unbalanced) does not harm the capabilities of the model, but only has an effect in some performance metrics. In this sense, the problem of imbalance in the test set is easily solved by using suitable metrics, such as the Kappa or A_{mean}, that do not conceal the errors for the minority class (Molina-Coronado et al., 2020).

Table 12 shows the results obtained for detectors under this scenario. The use of unbalanced data for training results in a reduction in the proportion of correctly classified malware (TPR), with respect to the results obtained using the baseline configuration (see Table 5). Imbalance in the training data is translated into less malware information provided to algorithms, making it difficult for detectors to learn the characteristics of this class of samples. On average the performance of most methods decreased about 12% as measured by their A_{mean} values. Among the best scoring methods in the baseline scenario, MaMaDroid is one of the methods that suffered a more significant decrease in this scenario, with a reduction of 18% in its A_{mean} value.

7.5. Evasion scenario

This scenario is devoted to test the robustness of ML-based malware detectors under conditions where attackers attempt to bypass detection using obfuscation. Our aim with this section is to illustrate the need of carrying out the security analysis of detectors. There are many evasion techniques targetting the different phases of the malware detection process that have not been included in this study to keep the size and scope manageable, but deserve further analysis, e.g., sandbox detection (Vidas and Christin, 2014) or adversarial attacks against ML models (Papernot et al., 2017). We focus our evaluation in obfuscation[9] because it is a classical evasion technique that is commonly overlooked by the authors of proposals (Hammad et al., 2018). We selected a set of obfuscation strategies that are commonly used in the wild to hide Android malware behaviors (Dong et al., 2018). For each obfuscation strategy considered, as well as for the combination of all of them, a new obfuscated dataset is obtained by: (1) randomly sampling 10% of the apps from 2012 to 2015 in our master dataset and, (2) applying the required transformations to such apps using the ObfusAPK (Aonzo et al., 2020) and the AAMO (Preda and Maggi, 2017) tools. These transformations are:

- Renaming. The original name and identifiers of user-defined classes, fields and methods are changed by meaningless strings.
- Changes in the structure of the code. This form of obfuscation includes: (1) call indirections to add an intermediate function that calls the function originally present in the code, (2) insertion of *goto* instructions, (3) inversion of conditionals to modify the execution flow of the app, (4) insertion of junk code, and (5) reflection[10] to hide function calls to internal code and to the Android framework (APIs).
- Encryption. This technique involves: (1) the generation of a encryption/decryption random key, (2) the encryption of native libraries, strings and assets, and (3) the insertion of a decryption code which is called from every part of the app where these resources are requested.

The results are summarized in Table 13. On average, when considered separately, the most successful strategies for evading detection are changes in code structure and encryption. However, generally speaking, individual obfuscation strategies are not as effective as a combination of all of them. Most detectors are prone to misclassify obfuscated malware as goodware, as shown by the decrease in their TPR values for the combined scenario. In this regard, the least affected model is Drebin, being able to identify 79% of the obfuscated malware, with 13% false positives. Others, such as DroidDet and MaMaDroid obtained more moderate performance figures, both identifying around 70% of the obfuscated samples according to their A_{mean} values when using the three obfuscation techniques in combination.

7.6. Evolution scenario

Both malware and goodware evolve over time, i.e., do not follow a stationary distribution. Therefore, it is logical to think that static detectors trained with apps for a certain period of time will not necessarily work well with more recent apps. To prove this assumption, the models trained using the balanced configuration (with data from the period 2012–2015) are tested with goodware and malware obtained between 2016 and 2019.

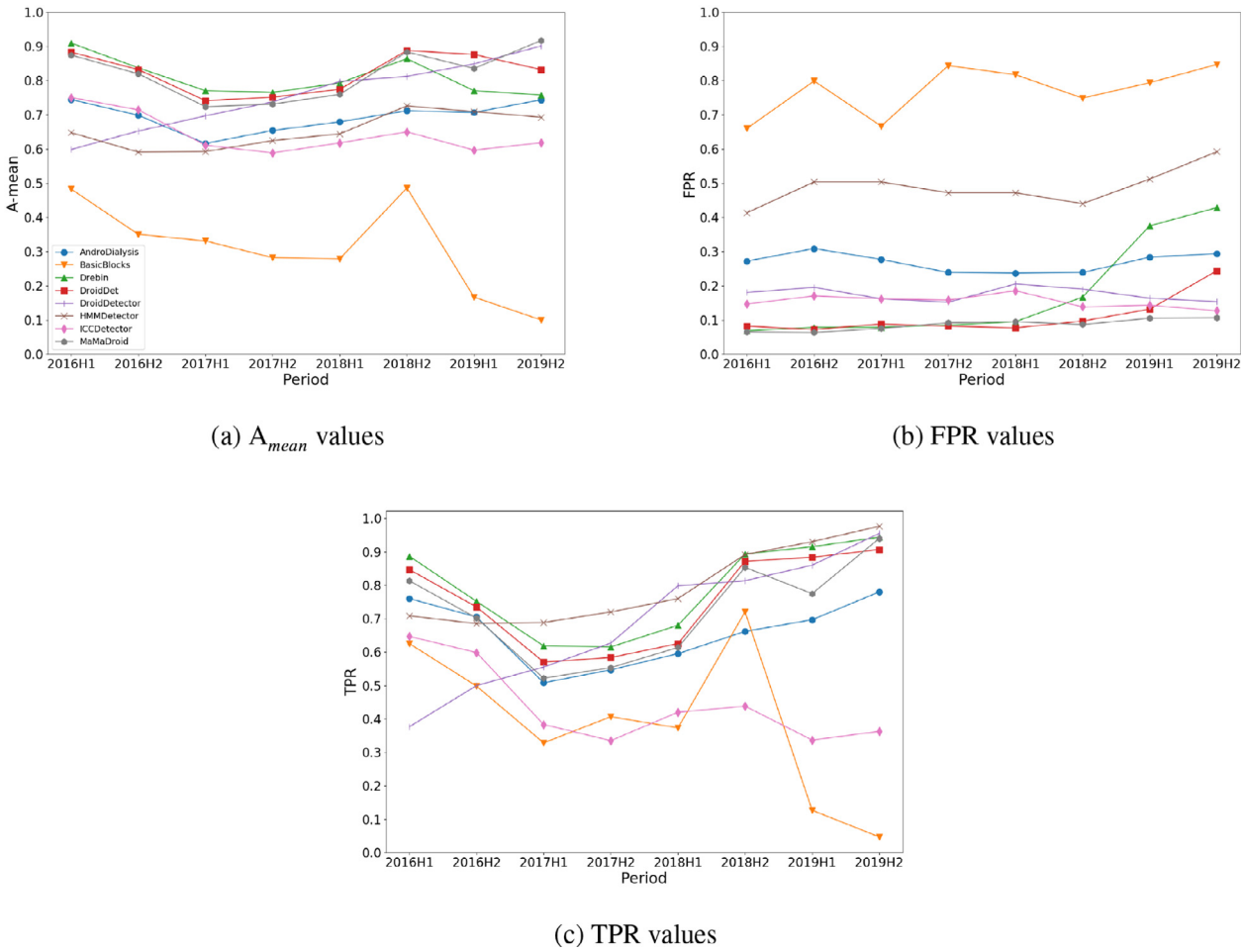
The overall performance of most of the methods under this scenario is characterized by an increment of both false positives and false negatives (see Table 14). Fig. 4a provides a more detailed view of the evolution of the TPR, FPR and A_{mean} values throughout the period between 2016 and 2019. As can be seen, the performance of detectors changes in a notable manner from one period to another. Our hypothesis is that the popularity of some malware families decreases with time, and, at some point, they are replaced with newer families for which a detector may not have been trained. Also, at some point, an old behavior may become popular again, resulting in a sudden increase in the performance of that detector. This is why the lines in Fig. 4c do not follow clear, decreasing trends. In addition, contrary to what is stated in Pendlebury et al. (2019), the incremental trend in the number of false alarms of detectors (see Fig. 4b) indicates that goodware also changes its behavior over time. At any rate, our observations demonstrate the non-stationary nature of malware and goodware, and show how classic, batch-trained ML algorithms are not an appropriate solution for Android malware detection given the instability of their decisions.

[9] Obfuscation involves code modifications to hinder the static analysis of apps without affecting their functionality. From the point of view of malware creators, obfuscation hampers the extraction of the features that may be indicative of malicious behaviors in order to fool detectors (Suarez-Tangil and Stringhini, 2018).

[10] Reflection is a feature of some programming languages that allows an executing program to examine or "introspect" upon itself, and manipulate internal properties of the program.

Table 13
Performance of detectors with obfuscated apps for the evasion scenario.

	Renaming (Rn)			Changes in code structure (Co)			Encryption (Enc)			Rn+Co+Enc		
Method	TPR	FPR	A_{mean}	TPR	FPR	A_{mean}	TPR	FPR	A_{mean}	TPR	FPR	A_{mean}
AndroDialysis	0.793	0.288	0.752	0.376	0.072	0.652	0.431	0.074	0.678	0.459	0.086	0.686
BasicBlocks	0.624	0.163	0.73	0.211	0.631	0.29	0.418	0.137	0.64	0.172	0.095	0.538
Drebin	0.95	0.012	0.968	0.968	0.025	0.971	0.738	0.138	0.799	0.796	0.136	0.829
DroidDet	0.931	0.028	0.951	0.560	0.024	0.767	0.934	0.021	0.956	0.586	0.029	0.778
DroidDetector	0.154	0.216	0.469	0.049	0.102	0.473	0.133	0.126	0.503	0.157	0.211	0.473
HMMDetector	0.895	0.489	0.702	0.892	0.786	0.553	0.802	0.684	0.558	0.911	0.779	0.565
ICCDetector	0.246	0.033	0.606	0.667	0.023	0.822	0.668	0.023	0.822	0.279	0.024	0.627
MaMaDroid	0.733	0.034	0.848	0.737	0.042	0.845	0.92	0.037	0.941	0.533	0.048	0.743



(a) A_{mean} values (b) FPR values

(c) TPR values

Fig. 4. Evolution of the performance of baseline detectors for the period 2016–2019.

Table 14
Performance of baseline detectors using evaluation data between 2016 and 2019.

Method	TPR	FPR	Precision	F1	A_{mean}	Kappa
AndroDialysis	0.656	0.268	0.709	0.682	0.694	0.388
BasicBlocks	0.390	0.771	0.336	0.361	0.309	-0.381
Drebin	0.787	0.171	0.820	0.804	0.808	0.616
DroidDet	0.752	0.108	0.873	0.808	0.821	0.643
DroidDetector	0.685	0.175	0.796	0.736	0.755	0.510
HMMDetector	0.795	0.488	0.619	0.696	0.653	0.306
ICCDetector	0.440	0.153	0.741	0.552	0.643	0.286
MaMaDroid	0.721	0.086	0.893	0.798	0.817	0.635

8. Discussion: towards a realistic framework for malware detection

Based on the factors analyzed in the previous sections, this section presents a collection of ideas or recommendations to consider when designing and evaluating a realistic proposal for Android malware detection.

Undoubtedly, one of the main problems encountered when researching malware detectors for Android is the lack of reproducibility of many proposals in the literature. The datasets and code used in the experimental processes are rarely public, and the details provided in the papers are often not enough for a correct re-implementation of the methods. Therefore, the most important recommendation is that authors of future works ensure that their work and experimental processes are fully reproducible. In addi-

tion to this general aspect, the experiments conducted in the previous section have highlighted the importance of using adequate datasets when training models and assessing their performance. Indeed, as we have seen, depending on the datasets used, the same model can show near-perfect performances or be almost irrelevant. Thus, we can conclude that the datasets and experimental scenarios considered in the literature are unrealistic and should be revised.

Related to the previous statements, our analysis has shown that the presence of highly similar apps in the datasets influences the performance of models. This also opens the possibility to perform evasion attacks against classifiers, for example, by developing malware apps with snippets of code from minor malware variants. During training, models tend to focus on large groups of duplicates, instead of trying to generalize the variety of apps in the data. During model assessment, duplicates are the cause of misleading performance indicators that tend to overestimate the detection ability of models. We can conclude that training and evaluation efforts should leverage on datasets without duplicates to improve and demonstrate the generalization capacity of models (Hastie et al., 2009). Training should be carried out after balancing the representatives within each of the classes, i.e., between groups of similar samples; to avoid biases and exploit the generalization capacity of ML models. Also, contextual evaluations that take into account the prevalence of different malware families or groups are desirable to describe the actual reasons behind the performance of detectors. The removal of duplicates has an additional advantage, as the implicit reduction in the dimensionality of the data is useful to speed up the labeling, training and evaluation processes.

In relation to the labeling of the apps, we have seen that VirusTotal is widely used, but relying only on the VTD reported by this tool entails some risks (Zhu et al., 2020). To begin with, there is a lack of consensus on the labeling of the apps and on the inclusion of greyware in the training and evaluation of models. We should bear in mind that the line between malware and goodware is not clearly defined. As we demonstrated, the choice of the labeling criteria highly influences the generated model and the detection performance, simplifying the problem as the VTD is increased. This lack of consensus also results in the omission of a large amount of greyware which lies in between. However, such apps are present in the Android app ecosystem (Arp et al., 2020; Google, 2019a), so, realistic proposals should not ignore these type of apps but include them in their training and evaluation processes. The nature of such apps may be uncertain. In this regard, additional information such as the details included in the VirusTotal reports can also be used to support more sophisticated labeling techniques, identifying particular functionalities like advertising, bundling, etc. which could provide valuable information to the user about a given (greyware) app. This would help to create a taxonomy of the type of apps present in the Android ecosystem, and to reach consensus on what is goodware or malware. It would also help to determine the usefulness of a third category of “potentially unwanted apps” (Gorrie, 2022) and its consideration as a third class in the data. In addition, to avoid the use of fixed thresholds over the VTD for labeling, and as an alternative to binary or three-class classifiers, new detection proposals could explore regression models that provide a risk/maliciousness metric, multi-step learning approaches (Daoudi et al., 2022), unsupervised detectors that return a degree of dissimilarity with respect to the benign class (Mahindru and Sangal, 2021), or semi-supervised methods that do not require fully labeled datasets (Van Engelen and Hoos, 2020).

Also regarding labeling, a more general problem is that not all antivirus engines in VirusTotal are equally reliable, with some of them being correlated or specialized in specific types of malware (Miller et al., 2016). In addition, two engines from the same vendor but specific for different platforms may differ (Salem et al., 2021). Although reliable labels can be derived from the VTD (Zhu et al., 2020), setting simple thresholds to the VTD assumes that all antivirus engines are equally reliable in all situations and makes no distinctions between them. In order to overcome these aspects, techniques such as crowd learning (Whitehill et al., 2009), which measure the relevance of the different antivirus engines present in VirusTotal, could be used. Differences among vendors are even more noticeable when obtaining family labels from malware, since anti-virus firms do not share a common nomenclature. In this regard, despite that some useful tools are available for agreeing on family labels from VirusTotal reports (Sebastián and Caballero, 2020), we believe that there is still a lot of work to be done. For example, proposing a standard taxonomy of malicious behaviors that would allow a common specification of malware.

As a last problem with labeling, we have seen in the experimentation that the VTD, and thus, the labeling, changes over time (Zhu et al., 2020). Weakly labeled data and changes in the labeling can hinder the generation of robust classifiers, leading to detection errors. As a solution, the use of the VTD for labeling, on its own, should be abandoned and replaced by more sophisticated methods, for example, that incorporate informative features provided in the analysis reports of VirusTotal (Salem et al., 2021). Active learning may also be a useful tool in this sense. This technique allows the best representatives of each class to be selected, reducing the effort of manual labeling (Huang et al., 2010). Nonetheless, using sophisticated methods for labeling might become cumbersome and costly, and of course, their reliability should be validated.

Another important aspect is that malware and goodware apps are found in different proportions in the wild, depending on the source of the apps, but malware being the minority class (Pendlebury et al., 2019). Ideally, a detector should perform well when analyzing apps regardless of the source of the samples. Thus, proposals should be trained and tested assuming unbalanced datasets. Additionally, the adoption of suitable performance metrics for unbalanced scenarios, such as the kappa and A-mean metrics, should be contemplated as opposed to using others, such as the accuracy, that do not reflect the real performance of detectors in these contexts. We have seen that classical ML supervised classification algorithms used for malware detection do not properly manage imbalance, as they expect data to be equally balanced in order to build a robust model (He and Garcia, 2009). As such, the use and design of specific ML classifiers for unbalanced scenarios is an open research line, which could be promising in this area of study. Some examples include cost-based classification methods, and subsampling and oversampling methods (Fernández et al., 2018). In addition, depending on the particular scenario in which the model will be applied, and on the interest of the practitioner (for example, reducing the number of FPs), different loss functions could be used in the training phase. As alternative approaches, anomaly detection algorithms may also be considered (Molina-Coronado et al., 2020).

Next, detectors have to work in a hostile scenario where attackers will try to trick them to infect a system. These are one of the first lines of defence, so security of malware detectors is important. Despite that our evaluation was limited to simple evasion attacks based on the use of obfuscation techniques, we evidenced that most detectors are not designed with security in mind and are, in fact, vulnerable to attacks. Appropriate assessments of the robustness of detectors should consider all the steps involved in the malware detection process. Novel evaluation methodologies that take into account the latest technology and attacks are needed and represent a promising future line of research. These should consider the evaluation of approaches to attacks targeting the analysis phase of apps, such as sandbox detection (Yokoyama et al., 2016); or, to attacks that target ML algorithms, such as adversarial learning (Zhang and Li, 2019). In this regard, the proposal of robust

ML algorithms against novel attacks is a very interesting research line (Demontis et al., 2017). However, we should not forget that cutting-edge detectors will also need to counteract classical evasion attacks. Thus, the selection and design of feature sets that are most useful to detect obfuscated malware behaviors (Garcia et al., 2018), as well as, the improvement of app analysis tools, are still promising research areas. Therefore, obfuscated samples are also essential in testing datasets. As for evasion techniques and tools, in this paper we have considered only a few; however, a broader set of techniques and tools that implement them need to be thoroughly analyzed in future work.

Finally, the malware detection problem is non-stationary, i.e., classes evolve over time and rarely show constant characteristics. A realistic detector should be able to cope with the non-stationary nature of Android apps. We demonstrated that the analyzed detectors, which are based on classical ML classifiers, are not able to manage such changes. A simple solution to this limitation could be to apply periodic retraining procedures to obtain up-to-date detectors (Molina-Coronado et al., 2020). However, depending on the complexity of the algorithms, this process could increment the cost of building and maintaining detectors. Also, the cost of obtaining a sufficient number of labeled samples at a time should not be obviated. Some authors have focused on slowing-down the aging of models and reduce update efforts by finding similar API functions (Zhang et al., 2020). As a more promising ML alternative for dynamic scenarios, online or stream learning algorithms have been proposed (Narayanan et al., 2017), some of which weight the instances based on their prevalence or age. These algorithms can automatically manage the changes in the distribution of the data so that they are able to adapt the models without intervention whenever these changes occur (Lu et al., 2018). Accordingly, the suitability of these methods for malware detection needs to be explored further. Another option would be to consider anomaly detectors or hybrid misuse-anomaly detectors (Molina-Coronado et al., 2020) as they may be more capable of dealing with the evolution in the data and zero-day attacks.

9. Conclusion

One of the main conclusions of this work is that authors of ML-based Android malware detectors tend to be very optimistic when designing and evaluating their systems, ignoring factors such as the presence of duplicates in the datasets, the lack of robust labeling methods, the presence of greyware, the imbalance between malware and goodware, the existence of apps trying to evade detection and the evolution of apps. Our evaluation work has shown how these factors substantially affect the performance that can be achieved with the tested detectors. We have seen, therefore, that malware detectors are not ready for deployment in real environments. Another important problem we have studied is the lack of a common design and evaluation framework, stemming, among other things, from the unavailability of codes and standardized and appropriate datasets. This fact greatly complicates the reproducibility of experiments. Our contribution in this regard has been to release our data and codes so that they can be used by other researchers. Finally, we have included in this paper a number of ideas that can contribute to the design of malware detectors which are able to operate in realistic deployment scenarios.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Borja Molina-Coronado: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing. **Usue Mori:** Conceptualization, Methodology, Writing – review & editing. **Alexander Mendiburu:** Conceptualization, Methodology, Writing – review & editing. **Jose Miguel-Alonso:** Conceptualization, Methodology, Writing – review & editing.

Acknowledgments

This work has received support from the following programs: PID2019-104966GB-I00AEI (Spanish Ministry of Science and Innovation), IT-1504-22 (Basque Government), KK-2021/00095 and KK-2021/00065 (Elkartek projects SIGZE and ALUSMART supported by the Basque Government). Borja Molina-Coronado holds a predoctoral grant (ref. PRE_2020_2_0167) by the Basque Government.

References

Afianian, A., Niksefat, S., Sadeghiyan, B., Baptiste, D., 2019. Malware dynamic analysis evasion techniques: a survey. ACM Comput. Surv. (CSUR) 52 (6), 1–28.
Aghakhani, H., Gritti, F., Mecca, F., Lindorfer, M., Ortolani, S., Balzarotti, D., Vigna, G., Kruegel, C., 2020. When malware is packin'heat; limits of machine learning classifiers based on static analysis features. In: Proceedings of the Network and Distributed Systems Security (NDSS) Symposium 2020.
Alaa, M., Zaidan, A.A., Zaidan, B.B., Talal, M., Kiah, M.L.M., 2017. A review of smart home applications based on internet of things. J. Netw. Comput. Appl. 97, 48–65.
Allix, K., Bissyandé, T.F., Jérome, Q., Klein, J., Le Traon, Y., et al., 2016. Empirical assessment of machine learning-based malware detectors for android. Empir. Softw. Eng. 21 (1), 183–211.
Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., 2015. Are your training datasets yet relevant? In: Proceedings of the International Symposium on Engineering Secure Software and Systems. Springer, pp. 51–67.
Allix, K., Bissyandé, T.F., Klein, J., Le Traon, Y., 2016. Androzoo: collecting millions of android apps for the research community. In: Proceedings of the IEEE/ACM 13th Working Conference on Mining Software Repositories (MSR). IEEE, pp. 468–471.
Aonzo, S., Georgiu, G.C., Verderame, L., Merlo, A., 2020. Obfuscapk: an open-source black-box obfuscation tool for android apps. SoftwareX 11, 100403. doi:10.1016/j.softx.2020.100403.
Arlot, S., Celisse, A., 2010. A survey of cross-validation procedures for model selection. Stat. Surv. 4, 40–79.
Arora, A., Peddoju, S.K., Conti, M., 2019. Permpair: android malware detection using permission pairs. IEEE Trans. Inf. Forensics Secur. 15, 1968–1982.
Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K., 2020. Dos and don'ts of machine learning in computer security. arXiv preprint arXiv:2010.09470.
Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., Rieck, K., Siemens, C., 2014. Drebin: effective and explainable detection of android malware in your pocket. In: Proceedings of the NDSS, Vol. 14, pp. 23–26.
Arshad, S., Shah, M.A., Wahid, A., Mehmood, A., Song, H., Yu, H., 2018. Samadroid: a novel 3-level hybrid malware detection model for android operating system. IEEE Access 6, 4321–4339.
Canfora, G., Mercaldo, F., Visaggio, C.A., 2016. An hmm and structural entropy based detector for android malware: an empirical study. Comput. Secur. 61, 1–18.
Daoudi, N., Allix, K., Bissyandé, T. F., Klein, J., 2022. A two-steps approach to improve the performance of android malware detectors. arXiv preprint arXiv:2205.08265.
Demontis, A., Melis, M., Biggio, B., Maiorca, D., Arp, D., Rieck, K., Corona, I., Giacinto, G., Roli, F., 2017. Yes, machine learning can be more secure! a case study on android malware detection. IEEE Trans. Dependable Secur. Comput. 16 (4), 711–724.
Desnos, A., Gueguen, G., Bachmann, S., 2018. Androguard. [Online] Available: https://www.androguard.readthedocs.io/en/latest/.
Do, Q., Martini, B., Choo, K.-K.R., 2017. Is the data on your wearable device secure? an android wear smartwatch case study. Softw. Pract. Exp. 47 (3), 391–403.
Dong, S., Li, M., Diao, W., Liu, X., Liu, J., Li, Z., Xu, F., Chen, K., Wang, X., Zhang, K., 2018. Understanding android obfuscation techniques: a large-scale investigation in the wild. In: Proceedings of the International Conference on Security and Privacy in Communication Systems. Springer, pp. 172–192.
Faruki, P., Bharmal, A., Laxmi, V., Ganmoor, V., Gaur, M.S., Conti, M., Rajarajan, M., 2014. Android security: a survey of issues, malware penetration, and defenses. IEEE Commun. Surv. Tutor. 17 (2), 998–1022.
Feizollah, A., Anuar, N.B., Salleh, R., Suarez-Tangil, G., Furnell, S., 2017. Androdialysis: analysis of android intent effectiveness in malware detection. Comput. Secur. 65, 121–134.
Fernández, A., García, S., Galar, M., Prati, R.C., Krawczyk, B., Herrera, F., 2018. Learning From Imbalanced Data Sets, Vol. 10. Springer.

Garcia, J., Hammad, M., Malek, S., 2018. Lightweight, obfuscation-resilient detection and family identification of android malware. ACM Trans. Softw. Eng. Methodol. (TOSEM) 26 (3), 1–29.
Google, 2019a. Android security & privacy. 2018 year in review. https://www.source.android.com/security/reports/Google_Android_Security_2018_Report_Final.pdf.
Google, 2019b. Android security 2017 year in review. https://www.source.android.com/security/reports/Google_Android_Security_2017_Report_Final.pdf.
Gorrie, M., 2022. What is a pua (potentially unwanted application) or pup (potentially unwanted program)?[Online] Available: https://www.us.norton.com/internetsecurity-malware-what-are-puas-potentially-unwanted-applications.html.
Hammad, M., Garcia, J., Malek, S., 2018. A large-scale empirical study on the effects of code obfuscations on android apps and anti-malware products. In: Proceedings of the 40th International Conference on Software Engineering, pp. 421–431.
Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N.J., et al., 2020. Array programming with numpy. Nature 585 (7825), 357–362.
Hastie, T., Tibshirani, R., Friedman, J., 2009. The Elements of statistical Learning: Data Mining, Inference, and Prediction. Springer Science & Business Media.
He, H., Garcia, E.A., 2009. Learning from imbalanced data. IEEE Trans. Knowl. Data Eng. 21 (9), 1263–1284.
Huang, S.-J., Jin, R., Zhou, Z.-H., 2010. Active learning by querying informative and representative examples. Adv. Neural Inf. Process. Syst. 23, 892–900.
Kim, T., Kang, B., Rho, M., Sezer, S., Im, E.G., 2018. A multimodal deep learning method for android malware detection using various features. IEEE Trans. Inf. Forensics Secur. 14 (3), 773–788.
Labs, K., 2021. Mobile malware evolution 2020. [Online] Available: https://www.securelist.com/mobile-malware-evolution-2020/101029/.
Lashkari, A.H., Kadir, A.F.A., Taheri, L., Ghorbani, A.A., 2018. Toward developing a systematic approach to generate benchmark android malware datasets and classification. In: Proceedings of the International Carnahan Conference on Security Technology (ICCST). IEEE, pp. 1–7.
Li, L., Bissyandé, T.F., Papadakis, M., Rasthofer, S., Bartel, A., Octeau, D., Klein, J., Traon, L., 2017. Static analysis of android apps: a systematic literature review. Inf. Softw. Technol. 88, 67–95.
Li, Y., Jang, J., Hu, X., Ou, X., 2017. Android malware clustering through malicious payload mining. In: Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses. Springer, pp. 192–214.
Lindorfer, M., Volanis, S., Sisto, A., Neugschwandtner, M., Athanasopoulos, E., Maggi, F., Platzer, C., Zanero, S., Ioannidis, S., 2014. Andradar: fast discovery of android applications in alternative markets. In: Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Springer, pp. 51–71.
Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., Zhang, G., 2018. Learning under concept drift: a review. IEEE Trans. Knowl. Data Eng. 31 (12), 2346–2363.
Macario, G., Torchiano, M., Violante, M., 2009. An in-vehicle infotainment software architecture based on google android. In: Proceedings of the IEEE International Symposium on Industrial Embedded Systems. IEEE, pp. 257–260.
Mahindru, A., Sangal, A., 2021. Semidroid: a behavioral malware detector based on unsupervised machine learning techniques using feature selection approaches. Int. J. Mach. Learn. Cybern. 12 (5), 1369–1411.
Maiorca, D., Ariu, D., Corona, I., Aresu, M., Giacinto, G., 2015. Stealth attacks: an extended insight into the obfuscation effects on android malware. Comput. Secur. 51, 16–31.
Miller, B., Kantchelian, A., Tschantz, M. C., Afroz, S., Bachwani, R., Faizullabhoy, R., Huang, L., Shankar, V., Wu, T., Yiu, G., et al., 2016. Reviewer integration and performance measurement for malware detection. In: Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Springer, pp. 122–141.
Molina-Coronado, B., Mori, U., Mendiburu, A., Miguel-Alonso, J., 2020. Survey of network intrusion detection methods from the perspective of the knowledge discovery in databases process. IEEE Trans. Netw. Serv. Manag. 17 (4), 2451–2479.
Moser, A., Kruegel, C., Kirda, E., 2007. Limits of static analysis for malware detection. In: Proceedings of the Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). IEEE, pp. 421–430.
Narayanan, A., Chandramohan, M., Chen, L., Liu, Y., 2017. Context-aware, adaptive, and scalable android malware detection through online learning. IEEE Trans. Emerg. Top. Comput. Intell. 1 (3), 157–175.
Onwuzurike, L., Mariconti, E., Andriotis, P., Cristofaro, E.D., Ross, G., Stringhini, G., 2019. Mamadroid: detecting android malware by building Markov chains of behavioral models (extended version). ACM Trans. Privacy Secur. (TOPS) 22 (2), 1–34.
Papernot, N., McDaniel, P., Goodfellow, I., Jha, S., Celik, Z.B., Swami, A., 2017. Practical black-box attacks against machine learning. In: Proceedings of the ACM on Asia Conference on Computer and Communications Security, pp. 506–519.
Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E., 2011. Scikit-learn: machine learning in python. J. Mach. Learn. Res. 12, 2825–2830.
Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., Cavallaro, L., 2019. {TESSERACT}: eliminating experimental bias in malware classification across space and time. In: Proceedings of the 28th {USENIX} Security Symposium ({USENIX} Security 19), pp. 729–746.
Preda, M.D., Maggi, F., 2017. Testing android malware detectors against code obfuscation: a systematization of knowledge and unified methodology. J. Comput. Virol. Hacking Tech. 13 (3), 209–232.
Rastogi, V., Chen, Y., Jiang, X., 2014. Catch me if you can: evaluating android anti–malware against transformation attacks. IEEE Trans. Inf. Forensics Secur. 9 (1), 99–108.
Raudys, S.J., Jain, A.K., et al., 1991. Small sample size effects in statistical pattern recognition: recommendations for practitioners. IEEE Trans. Pattern Anal. Mach. Intell. 13 (3), 252–264.
Roy, S., DeLoach, J., Li, Y., Herndon, N., Caragea, D., Ou, X., Ranganath, V. P., Li, H., Guevara, N., 2015. Experimental study with real-world data for android app security analysis using machine learning. In: Proceedings of the 31st Annual Computer Security Applications Conference, pp. 81–90.
Salem, A., Banescu, S., Pretschner, A., 2019. Don't pick the cherry: an evaluation methodology for android malware detection methods. arXiv preprint arXiv:1903.10560.
Salem, A., Banescu, S., Pretschner, A., 2021. Maat: automatically analyzing virustotal for accurate labeling and effective malware detection. ACM Trans. Priv. Secur. (TOPS) 24 (4), 1–35.
Sebastián, S., Caballero, J., 2020. Avclass2: Massive malware tag extraction from av labels. In: Proceedings of the Annual Computer Security Applications Conference, pp. 42–53.
Souri, A., Hosseini, R., 2018. A state-of-the-art survey of malware detection approaches using data mining techniques. Hum. Centric Comput. Inf. Sci. 8 (1), 3.
Spreitzenbarth, M., Schreck, T., Echtler, F., Arp, D., Hoffmann, J., 2015. Mobile-sandbox: combining static and dynamic analysis with machine-learning techniques. Int. J. Inf. Secur. 14 (2), 141–153.
Statista, 2022. Mobile operating systems' market share worldwide from january 2012 to january 2022. [Online] Available: https://www.statista.com/statistics/272698/global-market-share-held-by-mobile-operating-systems-since-2009/.
Suarez-Tangil, G., Stringhini, G., 2018. Eight years of rider measurement in the android malware ecosystem: evolution and lessons learned. arXiv preprint arXiv:1801.08115.
Surendran, R., 2021. On impact of semantically similar apps in android malware datasets. ArXiv, abs/2112.02606.
Tam, K., Feizollah, A., Anuar, N.B., Salleh, R., Cavallaro, L., 2017. The evolution of android malware and android analysis techniques. ACM Comput. Surv. 49 (4), 1–41. (CSUR)
Tan, D.J., Chua, T.-W., Thing, V.L., 2015. Securing android: a survey, taxonomy, and challenges. ACM Comput. Surv. 47 (4), 1–45. (CSUR)
Van Engelen, J.E., Hoos, H.H., 2020. A survey on semi-supervised learning. Mach. Learn. 109 (2), 373–440.
Vidas, T., Christin, N., 2014. Evading android runtime analysis via sandbox detection. In: Proceedings of the 9th ACM Symposium on Information, Computer and Communications Security, pp. 447–458.
Wei, F., Li, Y., Roy, S., Ou, X., Zhou, W., 2017. Deep ground truth analysis of current android malware. In: Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA'17). Springer, Bonn, Germany, pp. 252–276.
Whitehill, J., Wu, T.-f., Bergsma, J., Movellan, J., Ruvolo, P., 2009. Whose vote should count more: optimal integration of labels from labelers of unknown expertise. Adv. Neural Inf. Process. Syst. 22, 2035–2043.
Xu, K., Li, Y., Deng, R.H., 2016. Iccdetector: ICC-based malware detection on android. IEEE Trans. Inf. Forensics Secur. 11 (6), 1252–1264.
Xu, M., Song, C., Ji, Y., Shih, M.-W., Lu, K., Zheng, C., Duan, R., Jang, Y., Lee, B., Qian, C., et al., 2016. Toward engineering a secure android ecosystem: a survey of existing techniques. ACM Computing Surveys 49 (2), 1–47. (CSUR)
Yokoyama, A., Ishii, K., Tanabe, R., Papa, Y., Yoshioka, K., Matsumoto, T., Kasama, T., Inoue, D., Brengel, M., Backes, M., et al., 2016. Sandprint: fingerprinting malware sandboxes to provide intelligence for sandbox evasion. In: Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses. Springer, pp. 165–187.
Yuan, Z., Lu, Y., Xue, Y., 2016. Droiddetector: android malware characterization and detection using deep learning. Tsinghua Sci. Technol. 21 (1), 114–123.
Zhang, J., Li, C., 2019. Adversarial examples: opportunities and challenges. IEEE Trans. Neural Netw. Learn. Syst. 31 (7), 2578–2593.
Zhang, X., Zhang, Y., Zhong, M., Ding, D., Cao, Y., Zhang, Y., Zhang, M., Yang, M., 2020. Enhancing state-of-the-art classifiers with API semantics to detect evolved android malware. In: Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, pp. 757–770.
Zhou, Y., Jiang, X., 2012. Dissecting android malware: characterization and evolution. In: Proceedings of the IEEE Symposium on Security and Privacy. IEEE, pp. 95–109.
Zhu, H.-J., You, Z.-H., Zhu, Z.-X., Shi, W.-L., Chen, X., Cheng, L., 2018. Droiddet: effective and robust detection of android malware using static analysis along with rotation forest model. Neurocomputing 272, 638–646.
Zhu, S., Shi, J., Yang, L., Qin, B., Zhang, Z., Song, L., Wang, G., 2020. Measuring and modeling the label dynamics of online anti-malware engines. In: Proceedings of the 29th USENIX Security Symposium (USENIX Security 20). USENIX Association, pp. 2361–2378.

Borja Molina-Coronado received his B.Sc. in Computer Engineering and his M.Sc. in Computer Science from the Technical University of Valencia and the University of the Basque Country UPV/EHU, in 2015 and 2017, respectively. He is a Ph.D. student in the Dept. of Computer Architecture and Technology of the UPV/EHU. His main research areas are malware analysis, machine learning and network security.

Usue Mori received her M.Sc. Degree in Mathematics, and a Ph.D. in Computer Science from the University of the Basque Country UPV/EHU, Spain, in 2010 and 2015, respectively. Since 2019, she has been working as a lecturer in the Dept. of Computer Science and Artificial Intelligence of the University of the Basque Country UPV/EHU. Her main research interests include clustering and classification of time series.

Alexander Mendiburu is a full professor at the Dept. of Computer Architecture and Technology of the University of the Basque Country UPV/EHU, where he has been working since 1999. He received his B.Sc. Degree in Computer Science and his Ph.D. Degree from the University of the Basque Country, Spain, in 1995 and 2006, respectively. His main research areas are evolutionary computation, time series, probabilistic graphical models, and parallel computing.

Jose Miguel-Alonso is a full professor at the Dept. of Computer Architecture and Technology of the University of the Basque Country UPV/EHU. Formerly, he was a Visiting Assistant Professor at Purdue University. He received his M.Sc. in Computer Science in 1989 and his Ph.D. in Computer Science in 1996, both from the UPV/EHU. He carries out research related to parallel and distributed systems, in areas such as network security, performance modeling and resource management in large-scale computing systems. Prof. Miguel-Alonso is a member of the IEEE Computer Society and of the HiPEAC European Network of Excellence.